IT 112: Web Systems & Technologies

BSIT-2B 2nd Sem

S.Y. 2025-2026

Final Project Proposal
Phase 2 Documentation for Quick Reserve: Online Bus & Travel Booking System

A Final Project by <Tech_ Vanguard /> (Week 11)

Submitted By:

Nathaniel O. Raynada

Anna Mae P. Broma

Camille T. Bercilla

Mariel O. Odoño

John Edmar L. Bongcodin

Justin James Dasallas

Submitted To:

Dr. Guillermo V. Red, Jr, DIT

April 2026

I. Overview of Backend Setup

The backend of the QuickReserve system is built using modern web technologies such as Node.js, Express.js, and MongoDB Atlas. These tools help the system run smoothly and handle data efficiently. The system is organized into different parts like configuration, models, controllers, and routes so that the code is clean, easy to understand, and easier to manage.

The main server is set up in server.js, where it handles requests from users. It uses middleware like CORS and JSON parsing to process data properly. The system connects to the database using a secure connection, allowing it to store and retrieve important information like user accounts, trip details, and bookings.

QuickReserve allows users to create an account, log in, search for trips, choose seats, and book tickets online. All of these actions are processed by the backend through different API routes. The system can also update or cancel bookings when needed, making it flexible for users.

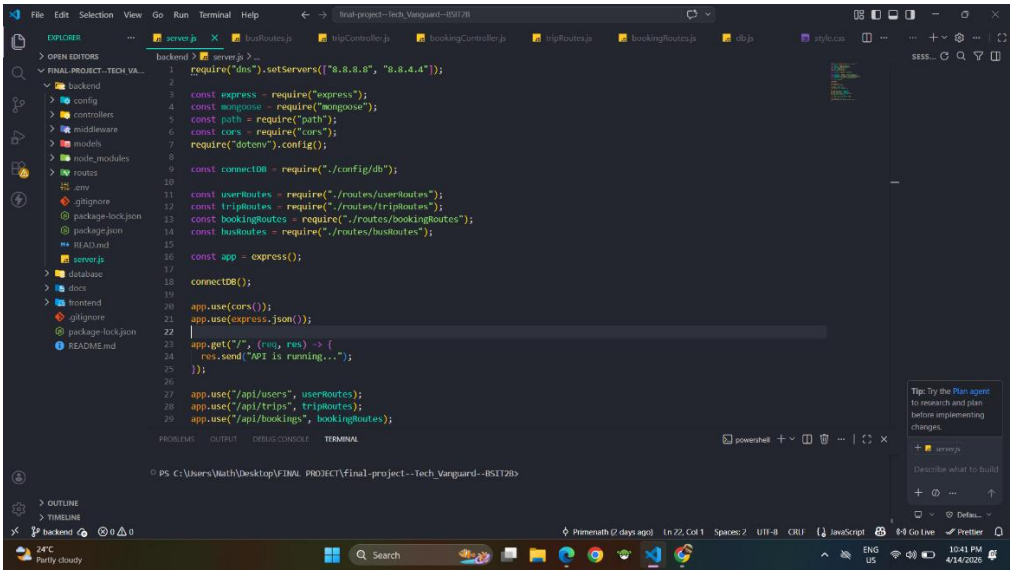
On the admin side, the system allows operators to manage trips, schedules, and passenger information. Admins can add, edit, or remove trip details and monitor bookings in real time. This helps keep the system organized and ensures everything runs properly.

To maintain better organization during development, the project uses a branching strategy in Git. Separate branches were created for **main**, **backend**, and **frontend** to avoid confusion when pushing and committing changes. This helps ensure that updates in each part of the system are isolated and properly managed without affecting the entire project. Once development is completed and all features are stable, the backend and frontend branches are merged into the main branch to produce the final version of the system.

In the end, QuickReserve backend is designed to be simple, reliable, and efficient. It helps make the booking process faster and more convenient for users while giving administrators full control over the system.

II. Code snippets of server.js, sample model, and routers

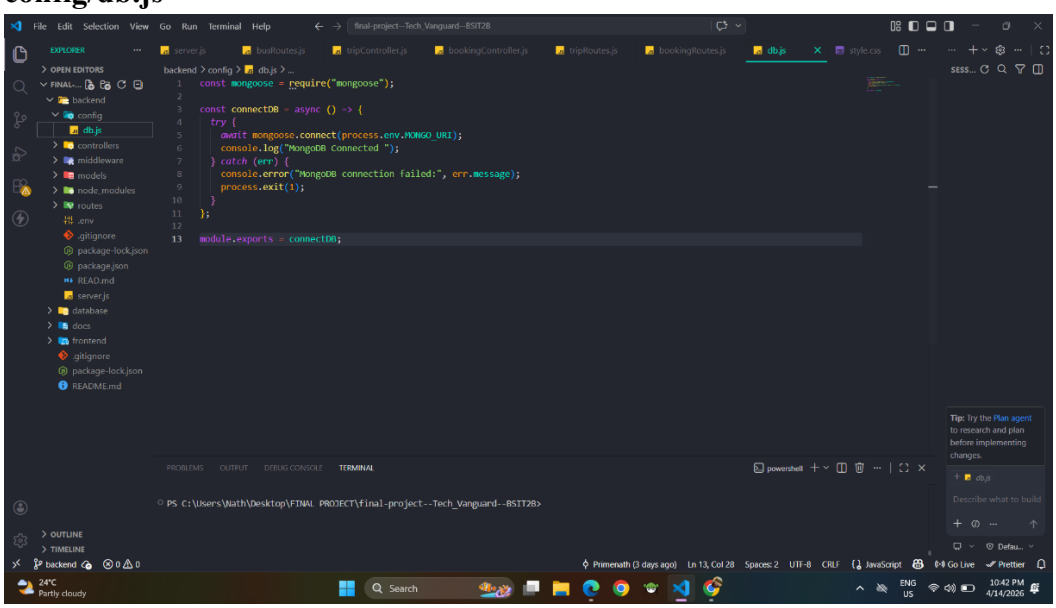
Server.js



```
require("dns").setServers(["8.8.8.8", "8.8.4.4"]);

const express = require("express");
const mongoose = require("mongoose");
const path = require("path");
const cors = require("cors");
require("dotenv").config();

const connectDB = require("./config/db");

const userRoutes = require("./routes/userRoutes");
const tripRoutes = require("./routes/tripRoutes");
const bookingRoutes = require("./routes/bookingRoutes");
const busRoutes = require("./routes/busRoutes");

const app = express();

connectDB();

app.use(cors());
app.use(express.json());

app.get("/", (req, res) => {
  res.send("API is running...");
});

app.use("/api/users", userRoutes);
app.use("/api/trips", tripRoutes);
app.use("/api/bookings", bookingRoutes);
```

Figure 1: Server Setup and API Route Configuration

config/db.js



```
const mongoose = require("mongoose");

const connectDB = async () => {
  try {
    await mongoose.connect(process.env.MONGO_URI);
    console.log("MongoDB Connected ");
  } catch (err) {
    console.error("MongoDB connection failed:", err.message);
    process.exit(1);
  }
};

module.exports = connectDB;
```

Figure 2: MongoDB Connection using Mongoose

Booking sample model

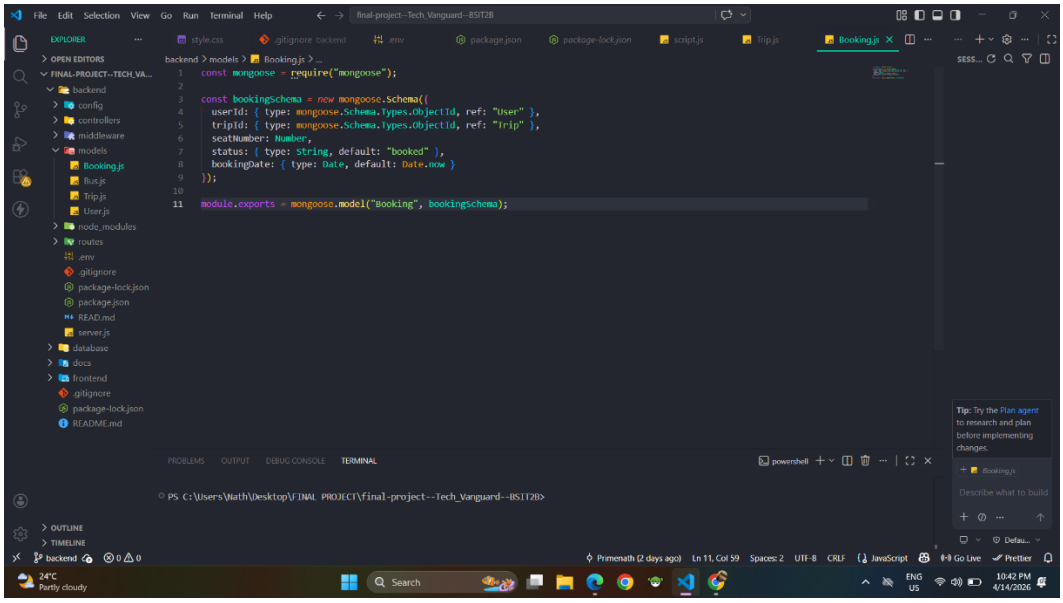


Figure 3: Booking Data Model

Admin routes

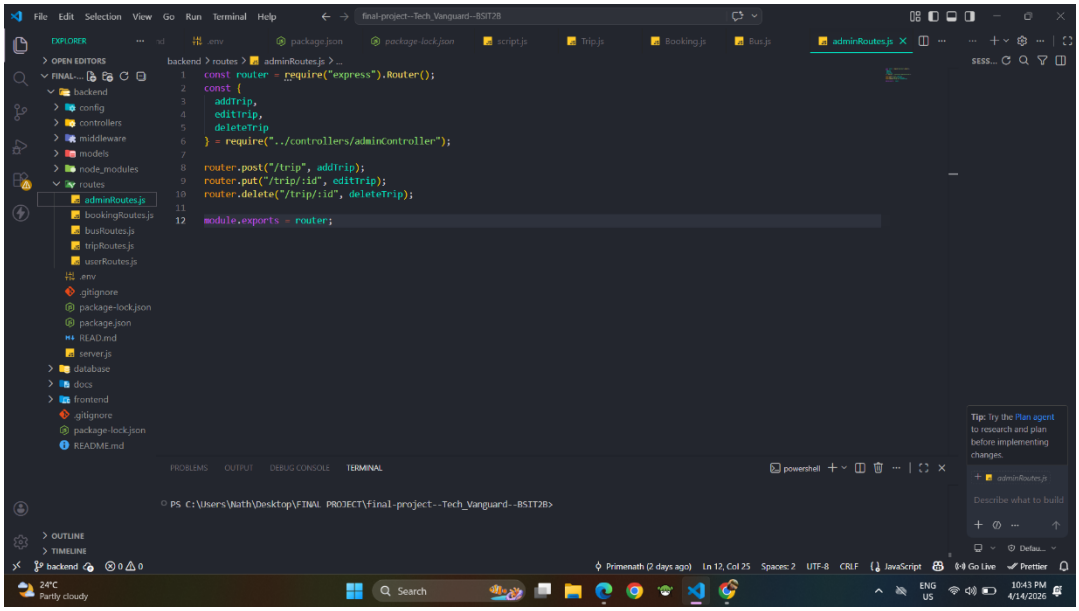


Figure 4: Admin managing system

User routes

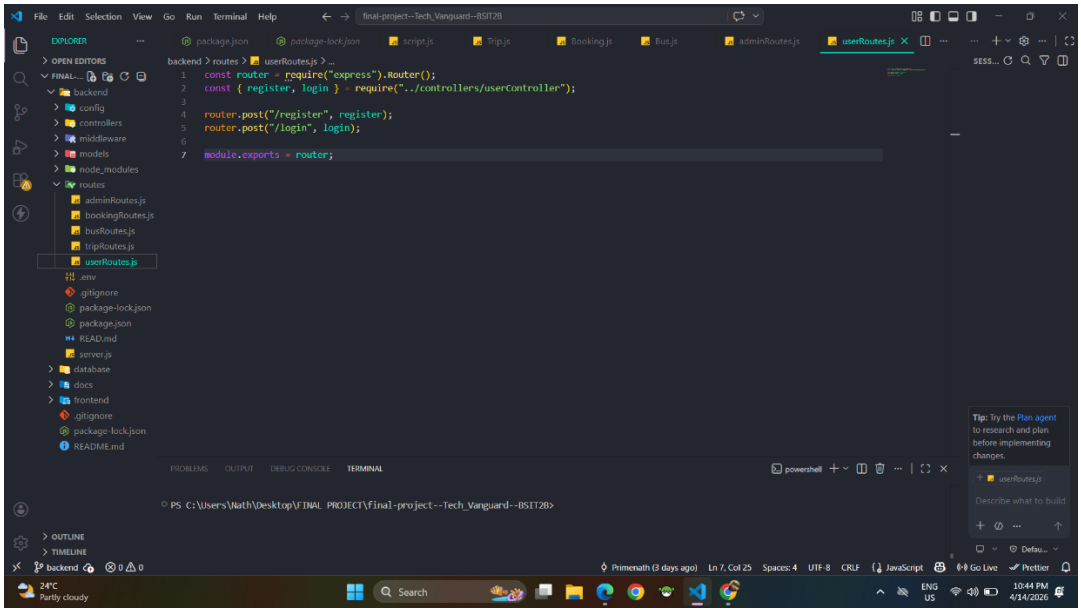
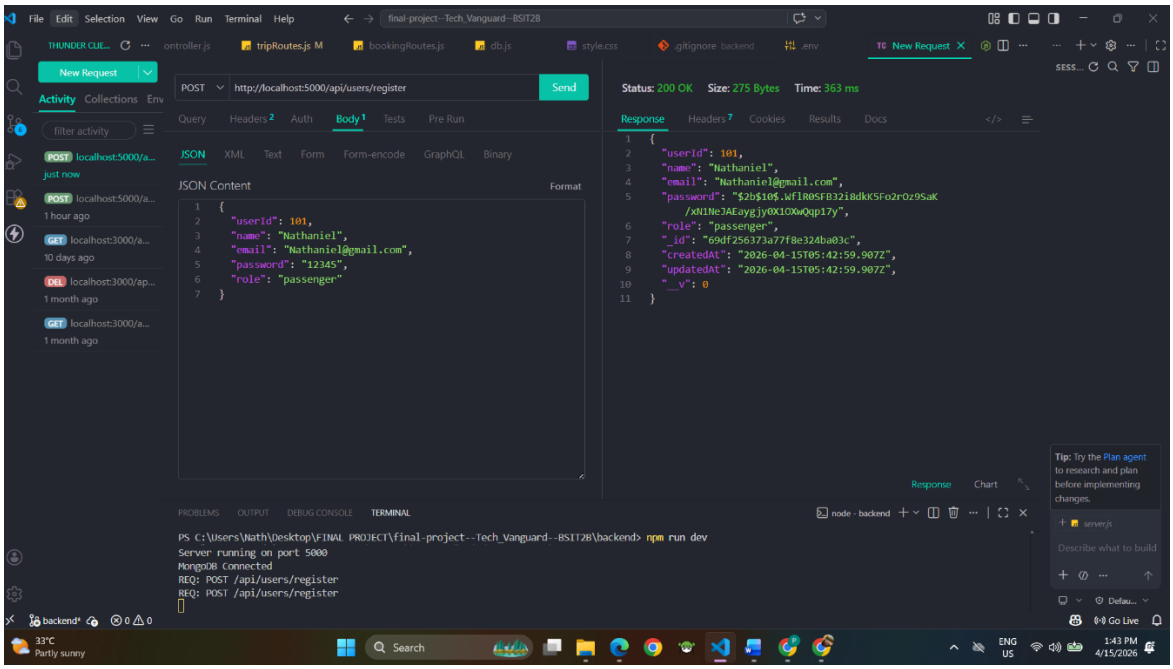


Figure 5: User use system

III. API Testing Screenshots

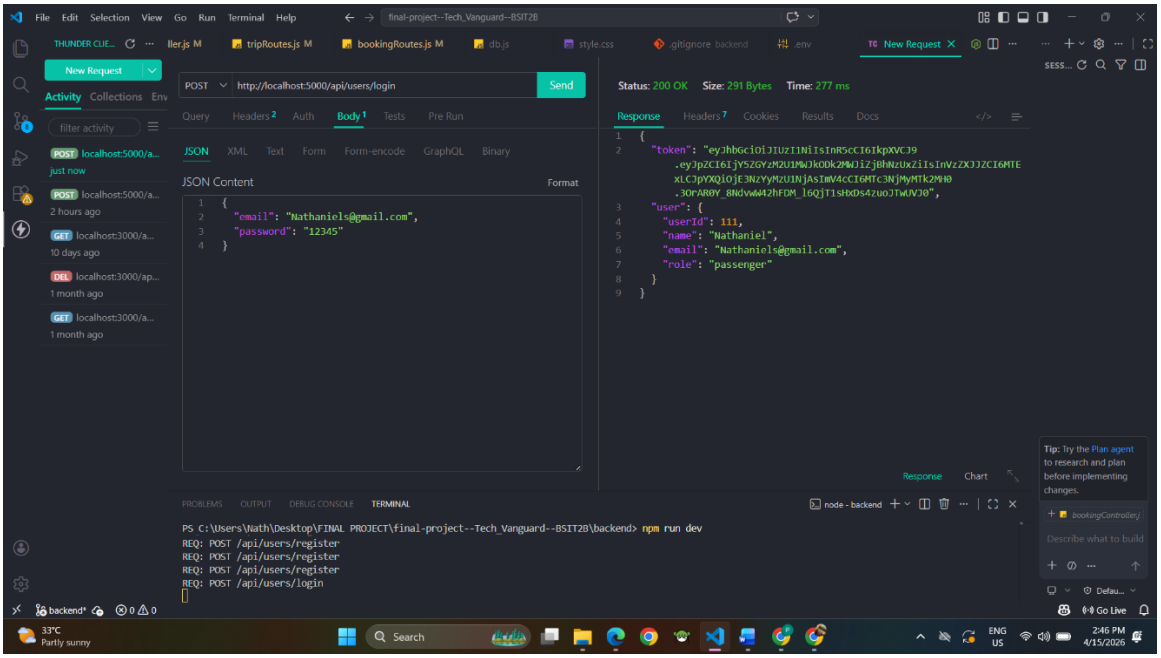
Figure 6: POST /api/users/register– Registers passenger

- Creates a new bus travel reservation in the QuickReserve system. The ouput will indicate time and data when does the user committed the reservation.



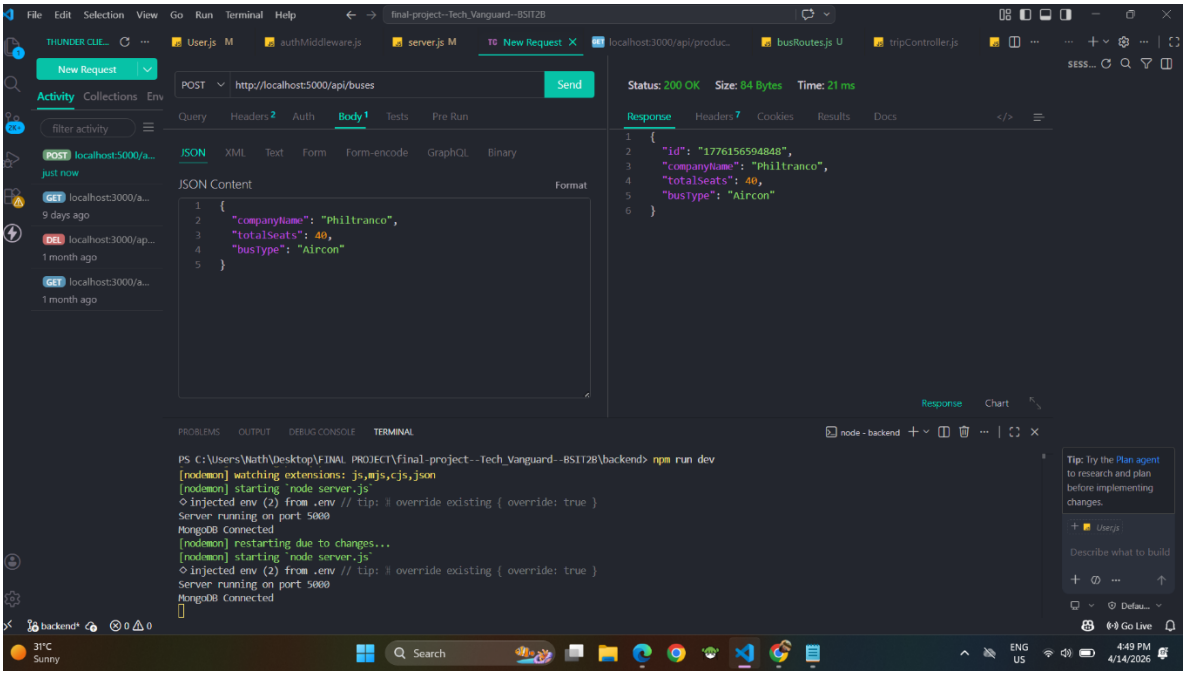
POST /api/users/login– Login passenger

- Logins on the account of that passenger



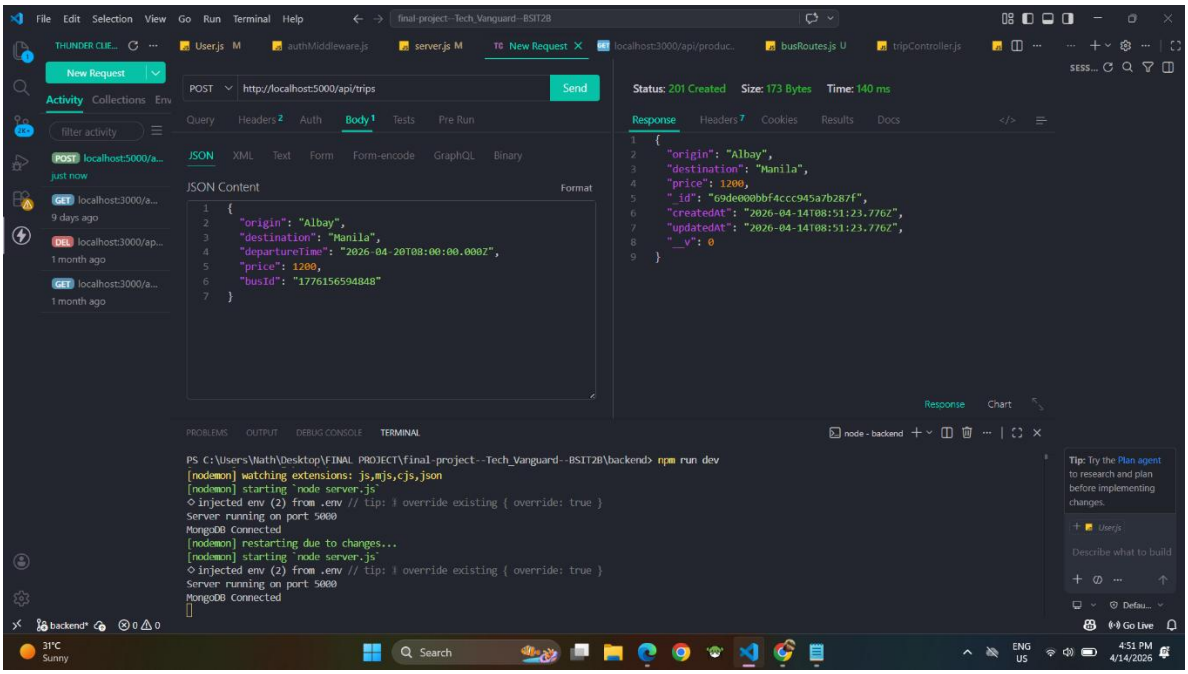
POST /api/buses– Book bus

- Phase where passenger book a bus and what kind of bus they prefer.



POST /api/trips– Where to go

- Phase where the passenger book a travel destination.



POST /api/bookings

- This phase indicates booking seats, user id, trip id and it indicates “confirmed” when the booking is done.

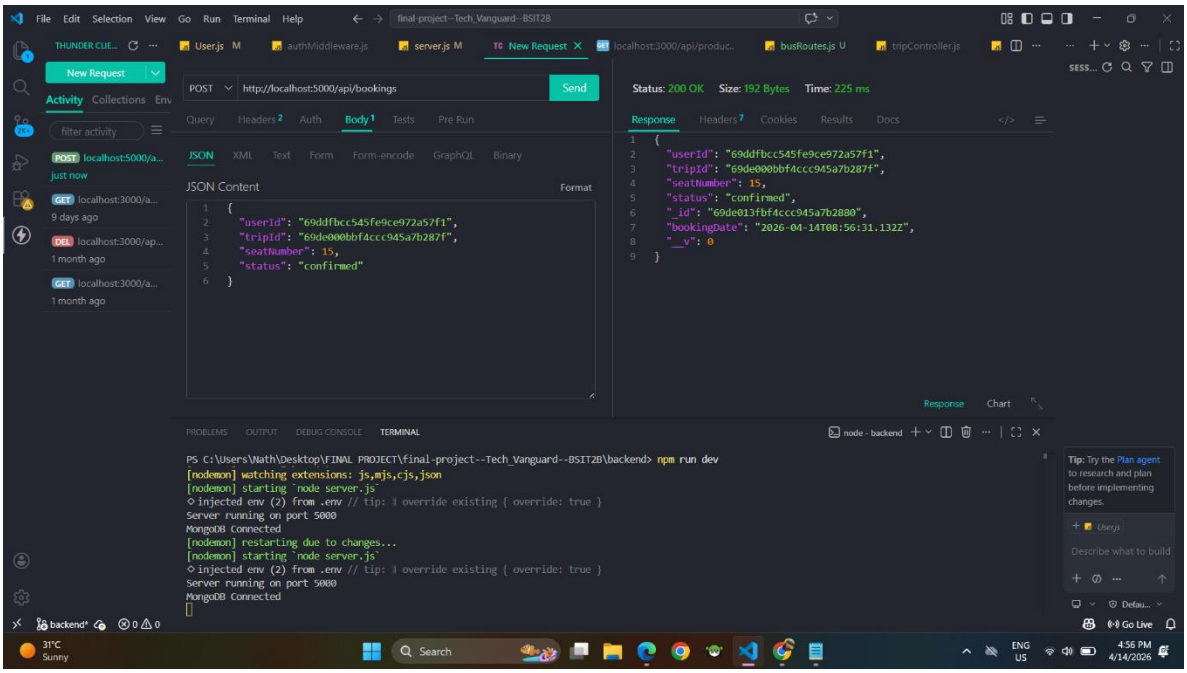
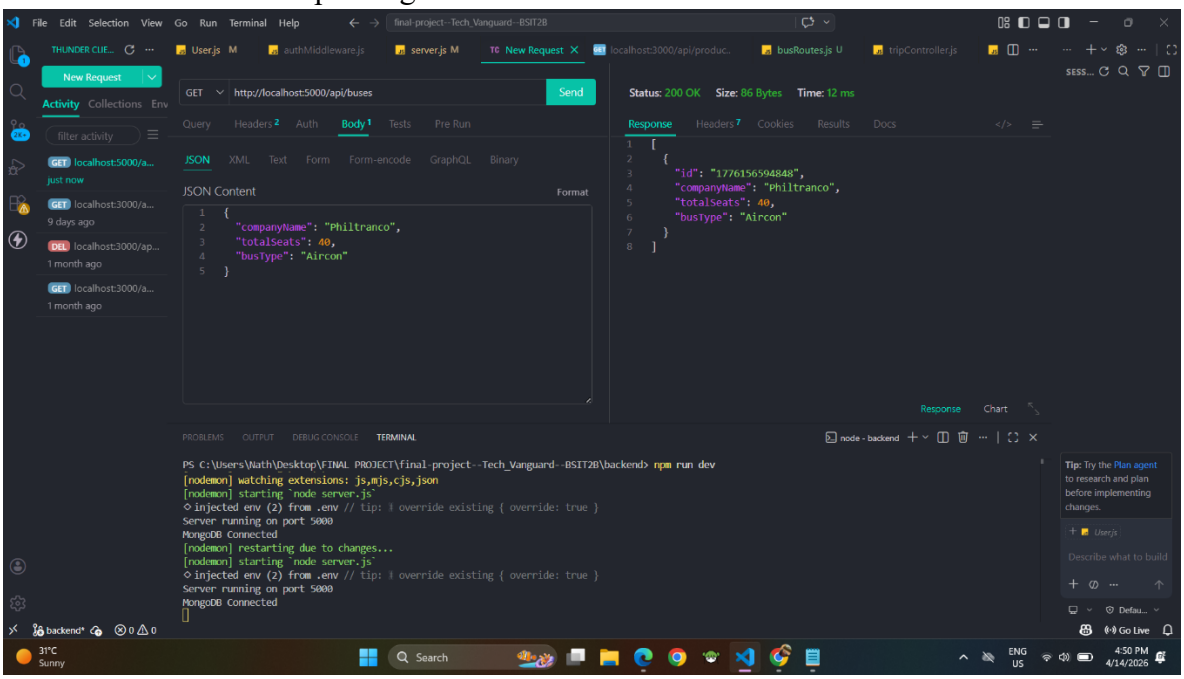


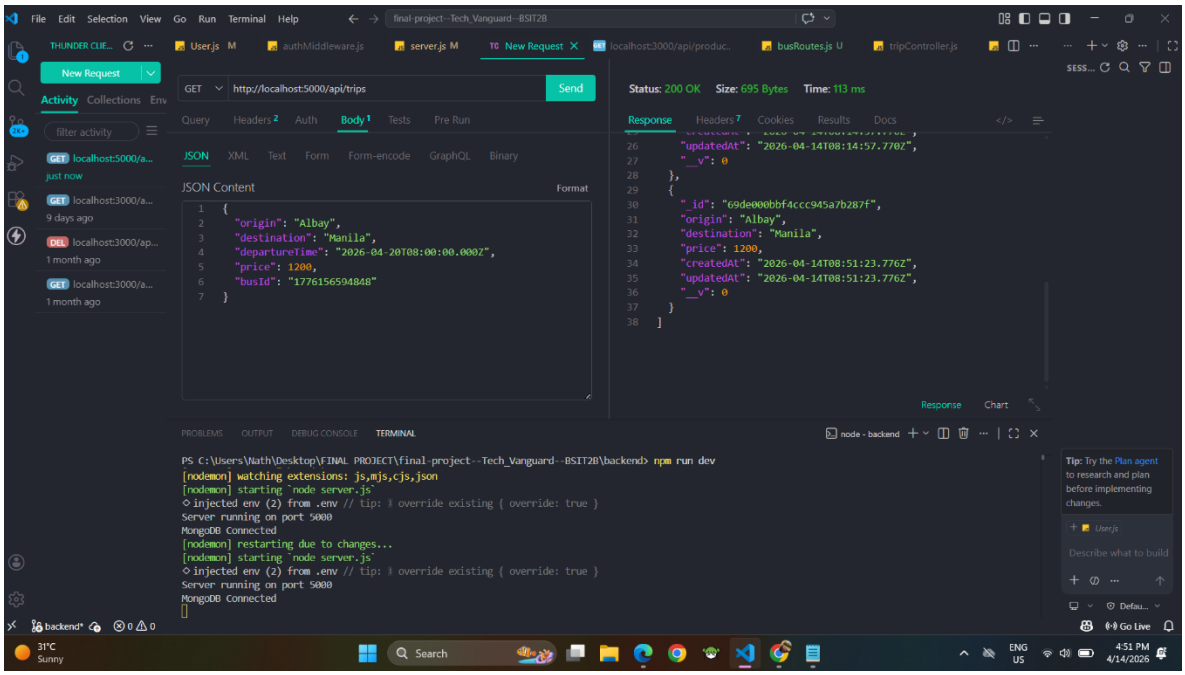
Figure 7: GET /Api/buses – Bus Booked

- Phase where the passenger views the booked bus



GET /Api/trips – See trips

Passenger sees the travel destination booked



GET /Api/bookings – View user bookings

Passenger sees the seat booking number

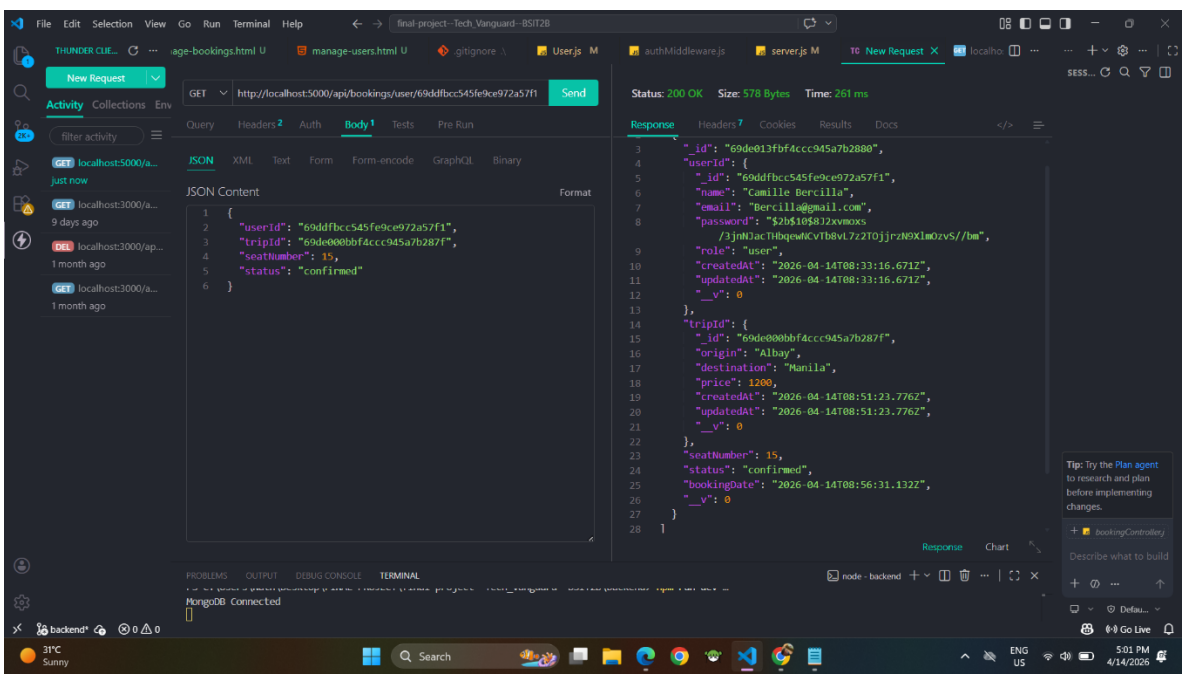


Figure 8: PUT /Api/bookings/:id – Update Booking

Updates an existing reservation, such as rescheduling due to unplanned events.

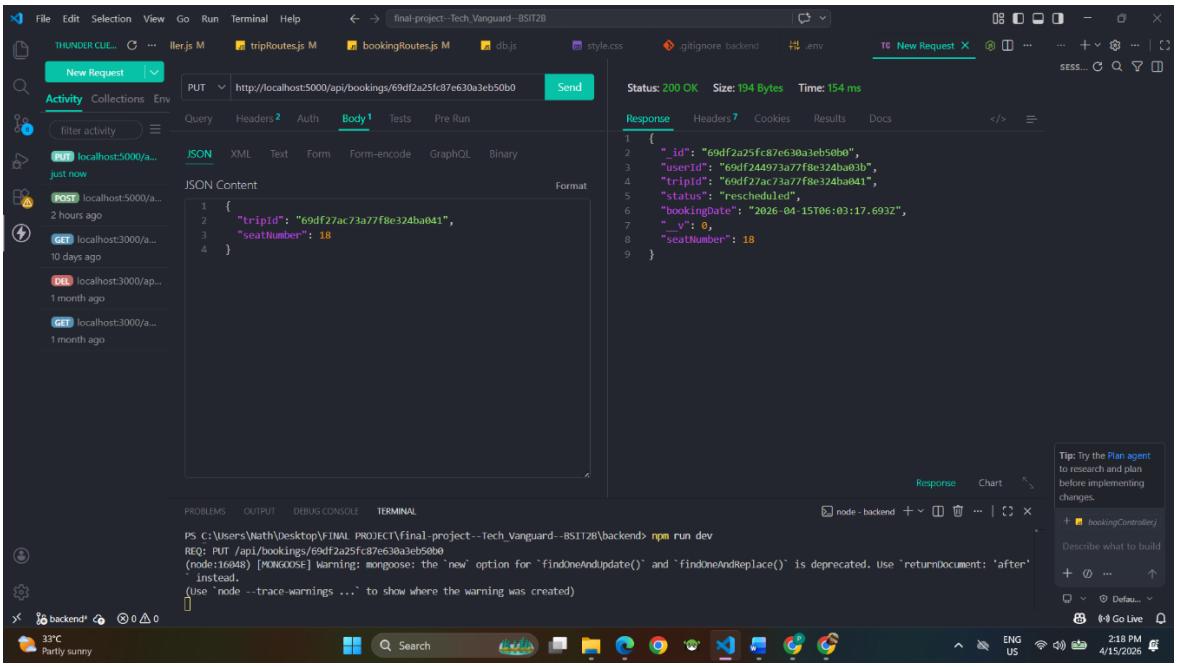
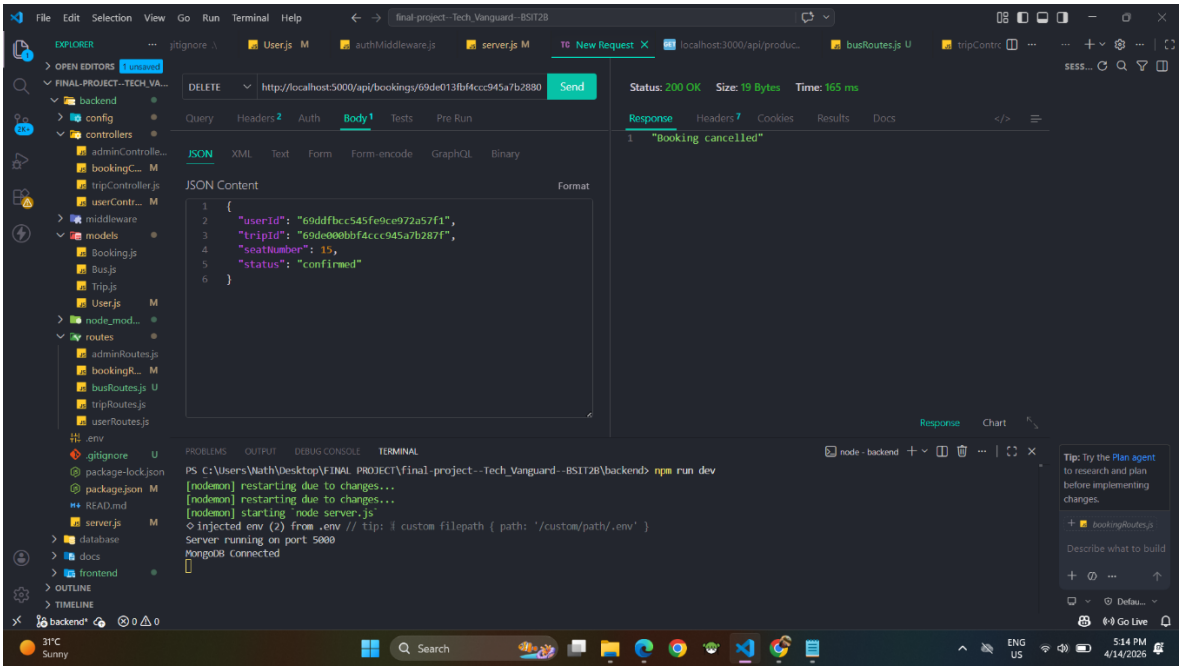
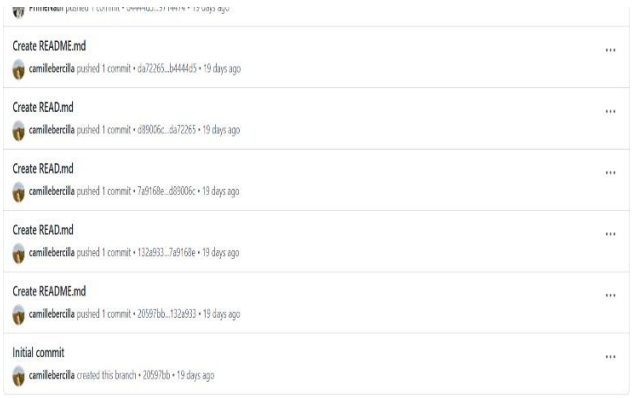
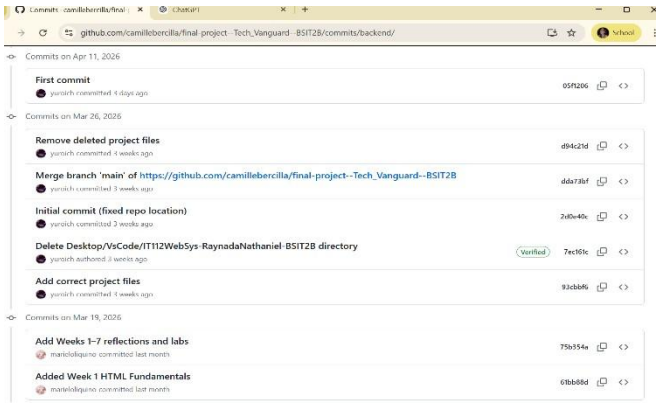


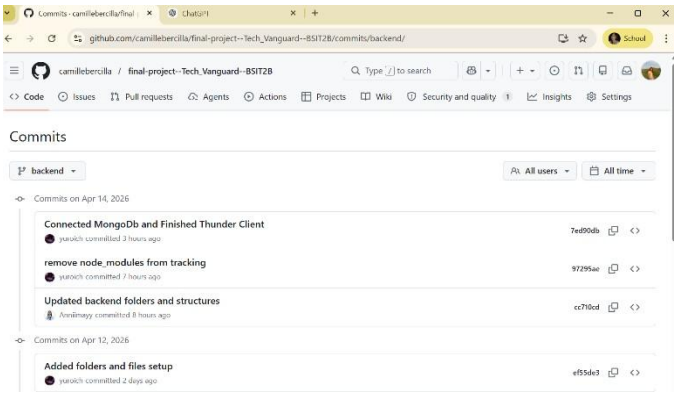
Figure 9: DELETE /Api/bookings/:id – Cancel Booking
Deletes or cancels a travel reservation from the system.



IV. Github Commits







V. Brief Explanation of each member's contribution

Each member of the group was assigned tasks to help complete the QuickReserve system. Through teamwork and cooperation, the group was able to complete the system, testing, and documentation. Everyone contributed in different ways to make sure the project was finished properly and on time.

Members:

Nathaniel O. Raynada – Project Manager/Front-End Developer

He assigned tasks to each member and guided the team throughout the project. He also contributed to both front-end and back-end development to support the system's progress. In addition, he helped in testing the system and assisted in connecting it to the database.

Anna Mae P. Broma – Back-End Developer

She worked on the backend of the system, including models and routes. She made sure that the system functions properly and that data is processed correctly. She also helped fix errors and improve the system's performance.

Camille Bercilla – GitHub Manager

She managed the project files in GitHub by organizing, updating, and uploading them properly. She made sure that all files were properly arranged and accessible to the team. She also helped in monitoring version control to avoid file conflicts.

Mariel O. Odoño – Database Manager

She handled database-related tasks and helped organize the documentation. She also ensured that the report follows the required format and is properly structured. In addition, she assisted in managing data storage and making sure information is accurate.

John Edmar Bongcodin – System Tester

He tested the system to ensure all features are working properly. He also checked for errors, bugs, and issues within the system. In addition, he reported problems to the team so they could be fixed and improved.

Justin Dasallas – Front-End Developer

He helped in documentation by capturing screenshots of the system and providing sample codes. He also ensured that the documentation matches the system's actual functionality. In addition, he assisted in checking the user interface to make sure it is clear and easy to use.